Tasks on the course Optimization Problem

Jianglin Lu
12. 01. 2019
Kazan Federal University
Optimization Problem

I. SOR method for convex problem with equality constraint

A. Preliminary

Finite difference methods:

In numerical analysis, finite-difference methods (FDM) are discretizations used for solving differential equations by approximating them with difference equations that finite differences approximate the derivatives.

Derivation of FDM from Taylor's polynomial:

Assuming the function whose derivatives are to be approximated is properly-behaved, by Taylor's theorem, we can create a Taylor series expansion as follows:

$$f(x_0+h) = f(x_0) + \frac{f'(x_0)}{1!}h + \frac{f^2(x_0)}{2!}h^2 + \cdots + \frac{f^n(x_0)}{n!}h^n + R_n(x)$$

where $n!$ denotes the factorial of n, and $R_n(x)$ is a remainder term, denoting the difference between the Taylor polynomial of degree n and the original function. We will derive an approximation for the first derivative of the function f by first truncating the Taylor polynomial:

$$f(x_0+h) = f(x_0) + f'(x_0)h + R_1(x)$$

Setting $x_0 = a$, we have,

$$f(a+h) = f(a) + f'(a)h + R_1(x)$$

Dividing across by h gives:

$$\frac{f(a+h)}{h} = \frac{f(a)}{h} + f'(a) + \frac{R_1(x)}{h}$$

Solving for $f'(a)$:

$$f'(a) = \frac{f(a+h) - f(a)}{h} - \frac{R_1(x)}{h}$$

Assuming that $R_1(x)$ is sufficiently small, the approximation of the first derivative of f is as follows:

$$f'(a) \approx \frac{f(a+h) - f(a)}{h}$$

The Laplace operator:

The Laplace operator is a second order differential operator in the n-dimensional Euclidean space, defined as the divergence ($\nabla\cdot$) of the gradient (∇f). Thus if f is a twice-differentiable real-valued function, then the Laplacian of f is defined by

$$\Delta f = \nabla^2 f = \nabla\cdot\nabla f$$

where the latter notations derive from formally writing

$$\nabla = \left(\frac{\partial}{\partial x_1}, \cdots, \frac{\partial}{\partial x_n}\right)$$

Equivalently, the Laplacian of f is the sum of all the unmixed second partial derivatives in the Cartesian coordinates x_i:

$$\Delta f = \sum_{i=1}^{n} \frac{\partial^2 f}{\partial x_i^2}$$

The (continuous) Laplace operator in n-dimensions is given by $\Delta u(x) = \sum_{i=1}^{n} \partial_i^2 u(x)$. The discrete Laplace operator $\Delta_h u$ depends on the dimension n.

In 1D the Laplace operator is approximated as:

$$\Delta u(x) = u''(x) \approx \frac{u(x-h) - 2u(x) + u(x+h)}{h^2} =: \Delta_h u(x)$$

The 2D case shows all characteristics of the more general nD case. Each second partial derivative needs to be approximated similar to the 1D case:

$$\begin{aligned}
\Delta u(x,y) &= u_{xx}(x,y) + u_{yy}(x,y) \\
&\approx \frac{u(x-h,y) - 2u(x,y) + u(x+h,y)}{h^2} + \frac{u(x,y-h) - 2u(x,y) + u(x,y+h)}{h^2} \\
&= \frac{u(x-h,y) + u(x+h,y) - 4u(x,y) + u(x,y-h) + u(x,y+h)}{h^2} \\
&=: \Delta_h u(x,y)
\end{aligned}$$

which is usually given by the follow stencil:

$$\frac{1}{h^2}\begin{pmatrix} & 1 & \\ 1 & -4 & 1 \\ & 1 & \end{pmatrix}.$$

Description of the Methods of Jacobi, Gauss-Seidel, and Successive Over-Relaxation:
Give a linear system

$$Ax = f$$

The Jacobi iteration method for solving the above linear system is as follows:

$$x_i^{k+1} = \frac{1}{a_{ii}}\left(f_i - \sum_{j=1}^{i-1} a_{ij}x_j^k - \sum_{j=i+1}^{n} a_{ij}x_j^k\right)$$

or in the matrix form:

$$x^{k+1} = -D^{-1}(L+U)x^k + D^{-1}f$$

where $A = L + D + U$, the only nonzero entries in D are the diagonal entries in A, the only nonzero entries in L are entries in A below the the diagonal, and the only nonzero entries in U are entries in A above the diagonal.

The Gauss-Seidel method for solving the above linear system is as follows:

$$x_i^{k+1} = \frac{1}{a_{ii}}\left(f_i - \sum_{j=1}^{i-1} a_{ij}x_j^{k+1} - \sum_{j=i+1}^{n} a_{ij}x_j^k\right)$$

or in the matrix form:

$$x^{k+1} = -D^{-1}Lx^{k+1} - D^{-1}Ux^k + D^{-1}f$$

or

$$x^{k+1} = -(D+L)^{-1}Ux^k + (D+L)^{-1}f$$

The Successive Over-Relaxation (SOR) method for solving the above linear system is as follows:
First, we have:

$$\begin{aligned}
x^{k+1} &= -D^{-1}Lx^{k+1} - D^{-1}Ux^k + D^{-1}f \\
\Leftrightarrow x^{k+1} &= x^k + (-x^k - D^{-1}Lx^{k+1} - D^{-1}Ux^k + D^{-1}f)
\end{aligned}$$

Then, multiply the parameter of relaxation ω:

$$\begin{aligned}
x^{k+1} &= x^k + \omega(-x^k - D^{-1}Lx^{k+1} - D^{-1}Ux^k + D^{-1}f) \\
\Leftrightarrow x^{k+1} &= (D+\omega L)^{-1}((1-\omega)D - \omega U)x^k + \omega(D+\omega L)^{-1}f
\end{aligned}$$

B. Problem definition

Let the domain $\Omega = (0,1) \times (0,1)$, the so-called Dirichlet boundary value problem for Poisson equation is as follows:

$$\begin{cases} -\Delta u \equiv -\dfrac{\partial^2 u}{\partial t_1^2} - \dfrac{\partial^2 u}{\partial t_2^2} = f & \text{on } \Omega, \\ u = 0 & \text{on } \partial\Omega. \end{cases} \tag{1}$$

where f is selected from $f(t) = \sin \pi t_1 \sin \pi t_2$ and $f(t) = \{\frac{1}{h^2}$ for $t_{ij} = (\frac{1}{2}; \frac{1}{2}); 0$ otherwise $\}$.

We approximate this problem by using the finite difference method in the next section.

C. Solution of Dirichlet boundary value problem for Poisson equation

Define a uniform mesh $\bar{\omega} = \{t = t_{ij} = (ih, jh) : 0 \leqslant i, j \leqslant m+1,\ (m+1)h = 1\}$ on the closure $\bar{\Omega} = \Omega \cup \partial\Omega$ of the domain. The set of the internal nodes will be denoted as ω, while Γ is the set of the nodes on $\partial\Omega$.

Below we consider mesh functions $u : \bar{\omega} \to \mathbb{R}$. A value of a mesh function $u(t)$ at the mesh point t_{ij} we denote by u_{ij}: $u(t_{ij}) = u_{ij}$.

The partial derivatives of a function $u(t)$ at a mesh point $t \in \omega$ can be approximated by "left" or "right" finite differences (difference quotients), namely,

$$\bar{\partial}_1 u(t_{ij}) = \frac{u_{ij} - u_{i-1,j}}{h},\ \partial_1 u(t_{ij}) = \frac{u_{i+1,j} - u_{ij}}{h}$$

and similarly defined $\overline{\partial}_2 u_i,\ \partial_2 u_i$.

By Δ_h we denote mesh Laplace operator, so $\Delta_h u(t) = \bar{\partial}_1 \partial_1 u(t) + \bar{\partial}_2 \partial_2(t)$ for a point $t \in \omega$.

Finite difference scheme for (1):

$$\begin{gathered} -\partial_1 \bar{\partial}_1 u(t) - \partial_2 \bar{\partial}_2 u(t) = f(t), \quad \text{for } t = t_{ij} \in \omega, \\ u(t) = 0, \quad \text{for } t \in \partial\omega, \end{gathered}$$

or in "indices form":

$$\begin{gathered} \frac{1}{h^2}\big(4u_{ij} - u_{i-1j} - u_{i+1j} - u_{ij-1} - u_{ij+1}\big) = f_{ij} \text{ for } 1 \leqslant i, j \leqslant m; \\ u_{0j} = u_{m+1,j} = u_{i0} = u_{i,m+1} = 0. \end{gathered} \tag{2}$$

Problem (2) has dimension $n = m^2$, and its unknowns are $u_{ij},\ i, j = 1, 2, \ldots, m$. To write this problem in a matrix-vector form we need to present the set $\{u_{ij}\}$ in the form of a vector from $\mathbb{R}^n$. And this is equivalent to the choice of a enumeration (ordering) for the set of the mesh nodes ω. A traditional enumeration is so-called lexicographical one: "from left to right and from down to top". When using this enumeration one obtains a vector $y \in \mathbb{R}^n$ by the following rule:

$$y_1 = u_{1,1}, y_2 = u_{2,1}, \ldots, y_p = u_{m,1}, y_{m+1} = u_{1,2}, y_{m+2} = u_{2,2}, \ldots, y_{m^2} = u_{m,m}.$$

With lexicographical ordering for the set of the mesh nodes, the following symmetric and block-tridiagonal matrix $A \in \mathbb{R}^{n \times n}$ corresponds to the mesh Laplace operator $-\Delta_h$:

$$A = h^{-2} \begin{pmatrix} D & -E & 0 & \ldots & 0 \\ -E & D & -E & \ldots & 0 \\ \ldots & \ldots & \ldots & \ldots & \ldots \\ 0 & 0 & \ldots & D & -E \\ 0 & 0 & \ldots & -E & D \end{pmatrix},\ D = \begin{pmatrix} 4 & -1 & 0 & \ldots & 0 \\ -1 & 4 & -1 & \ldots & 0 \\ \ldots & \ldots & \ldots & \ldots & \ldots \\ 0 & 0 & \ldots & 4 & -1 \\ 0 & 0 & \ldots & -1 & 4 \end{pmatrix}, \tag{3}$$

where $D \in \mathbb{R}^{m \times m}$, E is unit $m \times m$ matrix.

Let vector $f \in \mathbb{R}^n$ corresponds to mesh function f_h, then system (2) becomes

$$Ay = f$$

with symmetric and positive definite matrix A. Also, because of the symmetry of A, it is equivalent to the problem of minimisation of the function $F(y) = \dfrac{1}{2}(Ay, y) - (f, y)$ over $y \in \mathbb{R}^n$.

Let $A = D - L - L^T$, where L is strictly low triangle submatrix of A and D is its diagonal part. Let also $\sigma \in (0, 2)$ be a relaxation parameter. SOR method for the equation $Ay = f$ is:

u^0 being an initial guess, find sequentially u^k from

$$\left(\frac{1}{\sigma} D - L\right) u^{k+1} \ni \left(\frac{1}{\sigma} - 1\right) D u^k + L^T u^k + f, \quad k = 0, 1, \ldots$$

$\Big($*Pointwise writing of the formula* (4)*: for* $i = 1, 2, \ldots, n$

$$u_i^{k+1} = (1-\sigma)u_i^k + \frac{\sigma}{a_{ii}}\Big(\sum_{j>i} a_{ij}u_j^{k+1} + \sum_{j<i} a_{ij}u_j^k + f_i\Big).\Big)$$

The details of the proposed algorithm can be seen in Table I as follows.

TABLE I: SOR method for solving problem (1)

Input: Mesh size m, function f.

Step 1: Initialize u randomly, set $k = m+2$, $h = 1/k$, $\sigma = 2/(1 + sin(pi * h))$ and $\varepsilon = h^2$.
Step 2: Set $u(1,:) = 0$, $u(k,:) = 0$, $u(:,1) = 0$, $u(:,k) = 0$, and $sum = \varepsilon + 1$.
Step 3: While $sum >= \varepsilon$
% 1. calculate new iteration u:
for $i = 2 : (k-1), j = 2 : (k-1)$
$u_{ij} = (1-\sigma)u_{ij} + \frac{\sigma}{4}(u_{i-1,j} + u_{i+1,j} + u_{i,j-1} + u_{i,j+1} + h^2 f_{ij})$.
End
% 2. calculate residual vector r:
for $i = 2 : (k-1), j = 2 : (k-1)$
$r_{ij} = \frac{1}{h^2}(4u_{ij} - u_{i-1,j} - u_{i+1,j} - u_{i,j-1} - u_{i,j+1}) - f_{ij}$.
End
% 3. Compare:
$sum = 0$;
if $sum = (\sum_{1 \leqslant i,j \leqslant m} h^2 r_{ij}^2)^{1/2} < \varepsilon$
break;
End
End

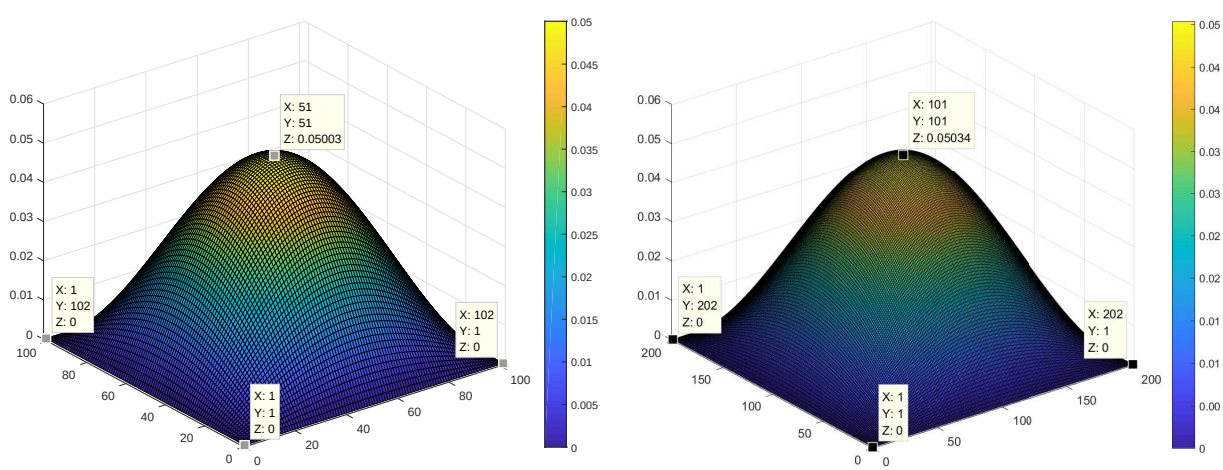
Output: u obtained by SOR method.

D. Experiments

(1) We take some experiments to indicate the effectiveness and efficiency of the above method. The selection of parameters are listed in the tables and the experimental results are illustrated in figures.

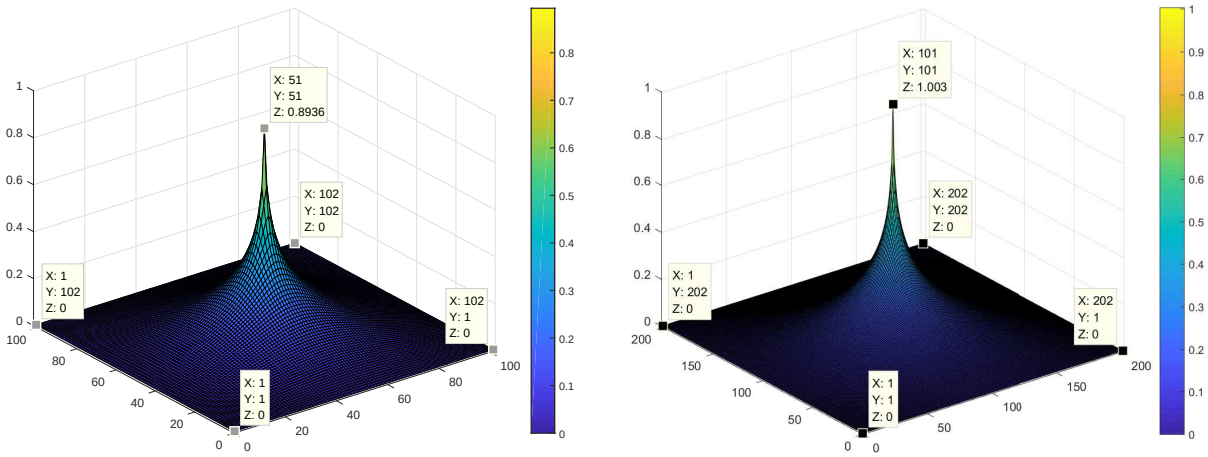
TABLE II: The selection of parameters

	f	m	h	σ	ε
Experiment 1	$f(t) = \sin \pi t_1 \sin \pi t_2$	100	$1/(m+2)$	$2/(1+sin(\pi h))$	h^2
Experiment 2	$f(t) = \sin \pi t_1 \sin \pi t_2$	200	$1/(m+2)$	$2/(1+sin(\pi h))$	h^2
Experiment 3	$f(t) = \{\frac{1}{h^2}$ for $t_{ij} = (\frac{1}{2}; \frac{1}{2}); 0$ otherwise$\}$	100	$1/(m+2)$	$2/(1+sin(\pi h))$	h^2
Experiment 4	$f(t) = \{\frac{1}{h^2}$ for $t_{ij} = (\frac{1}{2}; \frac{1}{2}); 0$ otherwise$\}$	200	$1/(m+2)$	$2/(1+sin(\pi h))$	h^2



(a) (b)

Fig. 1: $u(t_1, t_2)$ obtained by SOR method (a) Experiment 1, (b) Experiment 2.



(a) (b)

Fig. 2: $u(t_1, t_2)$ obtained by SOR method (a) Experiment 3, (b) Experiment 4.

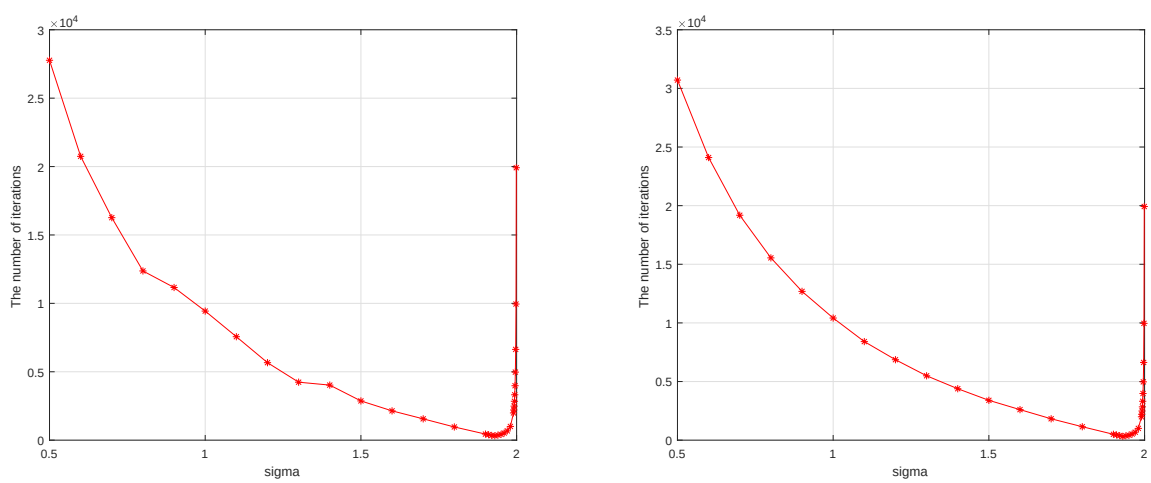
(2) We also take some experiments to show the relationship between the relaxation parameter σ and the number of iterations. The experimental results are listed in the Table III, IV, and shown in Fig.3.

TABLE III: The value of σ vs. the number of iterations, where $f(t) = \sin \pi t_1 \sin \pi t_2$, $\varepsilon = h^2$.

σ	0.5	0.6	0.7	0.8	0.9	1	1.1	1.2	1.3	1.4	1.5
iterations	27769	20747	16265	12370	11160	9435	7563	5665	4238	4025	2868
σ	1.6	1.7	1.8	1.9	1.91	1.92	1.93	1.94	1.95	1.96	1.97
iterations	2142	1558	965	444	401	345	322	371	421	521	679
σ	1.98	1.99	1.991	1.992	1.993	1.994	1.995	1.996	1.997	1.998	1.999
iterations	1002	1991	2211	2487	2844	3317	3979	4975	6634	9952	19915

TABLE IV: The value of σ vs. the number of iterations, where $f(t) = \{\frac{1}{h^2}$ for $t_{ij} = (\frac{1}{2}; \frac{1}{2}); 0$ otherwise$\}$, $\varepsilon = h^2$.

σ	0.5	0.6	0.7	0.8	0.9	1	1.1	1.2	1.3	1.4	1.5
iterations	30709	24102	19182	15546	12689	10416	8407	6862	5490	4387	3398
σ	1.6	1.7	1.8	1.9	1.91	1.92	1.93	1.94	1.95	1.96	1.97
iterations	2604	1823	1146	500	447	385	325	369	423	523	680
σ	1.98	1.99	1.991	1.992	1.993	1.994	1.995	1.996	1.997	1.998	1.999
iterations	1003	1993	2210	2488	2840	3318	3978	4973	6628	9962	19925



(a) (b)

Fig. 3: The value of σ vs. the number of iterations, (a) $f(t) = \sin \pi t_1 \sin \pi t_2$, $\varepsilon = h^2$, (b) $f(t) = \{\frac{1}{h^2}$ for $t_{ij} = (\frac{1}{2}; \frac{1}{2}); 0$ for other mesh points $\}$, $\varepsilon = h^2$.

E. Conclusions

We solve the so-called Dirichlet boundary value problem for Poisson equation by using the finite difference method. Since the matrix in the obtained linear system has the properties of high dimension, big condition number and sparsity, we consider using SOR iterative method to handle this problem. The conducted experiments indicate the effectiveness and efficiency of the used method. It is worth noting that, the selection of relaxation parameter σ has an important influence on the convergence. As the value of the parameter σ gradually increases (in the range $(0, 2)$), the number of iterations required for convergence decreases first and then increases sharply. To achieve the best convergence, it is significant to select an optimal parameter σ. As can be seen in Table III, IV, and shown in Fig.3, the optimal parameter σ in our experiments is approximated to 1.93, which leads to the minimum number of iterations.

F. Matlab codes

(1) Matlab code for solving $u(t_1, t_2)$:

```
clear all
%Initial
m = 100;
k = m+2;
h = 1/k;
sigma = 2/(1 + sin(pi*h));
varepsilon = h^2;
u = randn(k,k);
r = zeros(k,k);
u(1,:) = 0;
u(k,:) = 0;
u(:,1) = 0;
u(:,k) = 0;
sum = varepsilon+1;

while (sum ≥ varepsilon)
    % Step 1: Calculate new u
    for i = 2:(k-1)
        for j = 2:(k-1)
            u(i,j) = (1-sigma)*u(i,j)+(sigma/4)*(u(i-1,j)+u(i+1,j)+u(i,j-1)+u(i,j+1)+h^2*f(i,j,h));
        end
    end

    %Step 2: Calculate residual
    for i = 2:(k-1)
        for j = 2:(k-1)
            r(i,j) = (1/h^2)*( 4*u(i,j)-u(i-1,j)-u(i+1,j)-u(i,j-1)-u(i,j+1))-f(i,j,h);
        end
    end

    %Step 3: Compare
    sum = 0;
    for i = 2:(k-1)
        for j = 2:(k-1)
            sum = sum + h^2*r(i,j)^2;
        end
    end
    sum = sum^(1/2);
end
surf(u);

function value = f(i,j,h)
    value = sin(pi*i*h) * sin(pi*j*h);
    %if (i*h == 1/2 && j*h == 1/2)
    %   value = 1/(h^2);
    %else
    %   value = 0;
    %end
end
```

(2) Matlab code for comparing different values of σ:

```
clear all
%Initial
m = 100;
k = m+2;
h = 1/k;
varepsilon = h^2;
Tstep = [];
sigmaTest = [0.5,0.6,0.7,0.8,0.9,1,1.1,1.2,1.3,1.4,1.5,1.6,1.7,1.8,1.9,1.91,1.92,1.93,1.94,1.95,
             1.96,1.97,1.98,1.99,1.991,1.992,1.993,1.994,1.995,1.996,1.997,1.998,1.999];

for sigmaIter = 1:1:33
    step = 0;
    sigma = sigmaTest(sigmaIter);
    sum = varepsilon+1;
    u = randn(k,k);
    r = zeros(k,k);
    u(1,:) = 0;
    u(k,:) = 0;
    u(:,1) = 0;
    u(:,k) = 0;

    while (sum ≥ varepsilon)
        % Step 1: Calculate new u
        for i = 2:(k-1)
            for j = 2:(k-1)
                u(i,j) = (1-sigma)*u(i,j)+(sigma/4)*(u(i-1,j)+u(i+1,j)+u(i,j-1)+u(i,j+1)+h^2*f(i,j,h));
            end
        end

        %Step 2: Calculate residual
        for i = 2:(k-1)
            for j = 2:(k-1)
                r(i,j) = (1/h^2)*( 4*u(i,j)-u(i-1,j)-u(i+1,j)-u(i,j-1)-u(i,j+1))-f(i,j,h);
            end
        end

        %Step 3: Compare
        sum = 0;
        for i = 2:(k-1)
            for j = 2:(k-1)
                sum = sum + h^2*r(i,j)^2;
            end
        end
        sum = sum^(1/2);
        step = step+1;
    end
    Tstep = [Tstep step];
end
plot(sigmaTest(1:33), Tstep(1:33),'r-*');
grid on

function value = f(i,j,h)
    value = sin(pi*i*h) * sin(pi*j*h);
    %if (i*h == 1/2 && j*h == 1/2)
    %   value = 1/(h^2);
    %else
    %   value = 0;
    %end
end
```

II. SOR method for convex problem with inequality constraint

A. Preliminary

Finite difference approximation of Obstacle problem:

Let $K = \{u \in H_0^1(\Omega) : u(t) \geqslant 0 \text{ in } \Omega\}$ be a convex set in Sobolev space $H_0^1(\Omega)$ and $f \in L_2(\Omega)$ be a given function. Obstacle problem is the following variational inequality: find $u \in K$ such that

$$\int_\Omega \nabla u \cdot \nabla(v-u)dt \geqslant \int_\Omega f(t)(v-u)dt \quad \forall v \in K. \tag{4}$$

Variational inequality (4) has a unique solution u, which is at the same time the unique solution of the minimization problem:

$$u = \arg\min_{v\in K}\left\{J(v) = \frac{1}{2}\int_\Omega |\nabla v|^2 dt - \int_\Omega fvdt\right\} \tag{5}$$

Supposing f to be continuous, we can approximate the one-dimensional obstacle problem with $\Omega = (0,1)$ by a finite difference scheme on a uniform grid. We approximate the functional

$$J(u) = \frac{1}{2}\int_0^1 (u')^2 dt - \int_0^1 fudt \tag{6}$$

over the set $K = \{u(t) \geqslant 0 \text{ for } t \in (0,1),\ u(t) = 0, \text{ for } t = 0 \text{ and for } t = 1\}$.

Let $\bar{\omega} = \{t_i = ih : i = 0, 1, \ldots, n+1\}$, $(n+1)h = 1$ be a uniform mesh on the segment $[0,1]$ with meshsize $h > 0$, $u_i = u(t_i)(u_0 = u_{n+1} = 0)$ and $f_i = f(t_i)$. Functional J is approximated by a convex and differentiable (quadratical) function

$$J_h(u) = \frac{h}{2}\left(\frac{u_1^2}{h^2} + \sum_{i=1}^{n-1}\left(\frac{u_{i+1}-u_i}{h}\right)^2 + \frac{u_n^2}{h^2}\right) - h\sum_{i=1}^{n} f_i u_i, \tag{7}$$

When constructing this approximation, we use the difference quotients for the approximation of the first derivative

$$u'(t_i) \approx \frac{u_{i+1}-u_i}{h},\ u'(t_i) \approx \frac{u_i - u_{i-1}}{h}$$

and the quadrature rules for the approximation of the integrals

$$\int_0^1 F(t)dt \approx h\sum_{i=0}^{n} F(t_i), \qquad \int_0^1 F(t)dt \approx h\sum_{i=1}^{n+1} F(t_i)$$

for any continuous function $F(t)$.

The gradient of J_h is $\nabla J_h(u) = h(Au - f)$, where

$$A = h^{-2}\begin{pmatrix} 2 & -1 & 0 & \ldots & 0 & 0 \\ -1 & 2 & -1 & \ldots & 0 & 0 \\ 0 & -1 & 2 & \ldots & 0 & 0 \\ \ldots & \ldots & \ldots & \ldots & \ldots & \ldots \\ 0 & 0 & 0 & -1 & 2 & -1 \\ 0 & 0 & 0 & 0 & -1 & 2 \end{pmatrix}, \tag{8}$$

Denote by $K = \{u \in \Re^n : u_i \geqslant 0\ \forall i\}$ a convex set and by $(.,.)$ usual euclidian scalar product in $\Re^n$. The problem of the minimization of J_h over K is equivalent to variational inequality

$$(Au, v-u) \geqslant (f, v-u) \quad \forall v \in K \tag{9}$$

B. Problem definition

Variant 1: two-sided obstacle problem:

Let $\phi(x) \leqslant \psi(x)$ for all $x \in \Omega$ and $\phi(x) \leqslant 0 \leqslant \psi(x)$ for all $x \in \partial\Omega$. Define a closed and convex set

$$K = \left\{u \in H_0^1(\Omega) : \phi(x) \leqslant u(x) \leqslant \psi(x) \text{a.e.} \Omega\right\}$$

$$\int_\Omega \nabla u \cdot \nabla(v-u)dx \geqslant \int_\Omega f(v-u)dx \quad \forall v \in K, u \in K. \tag{10}$$

We approximate this problem by using the finite difference method in the next section.

C. Solution of Dirichlet boundary value problem for Poisson equation

Let $\Omega = (0,1)\times(0,1)$ and $\bar{\omega} = \{x = (ih, jh) : 0 \leqslant i, j \leqslant m+1,\ (m+1)h = 1\}$ be the uniform mesh on $\bar{\Omega}$.

The mesh scheme for the above obstacle problem is as follows:

$$-\partial_1\bar{\partial}_1 u(x) - \partial_2\bar{\partial}_2 u(x) + p(x, u(x)) \ni f(x), \quad \text{for} \quad x = x_{ij} \in \omega$$
$$u(x) = 0, \quad \text{for} \quad x \in \partial\omega$$

where $p(x, u(x)) \equiv p_{ij}(u_{ij})$ are the multivalued functions

$$p_{ij}(u_{ij}) = \begin{cases} (-\infty, 0], & \text{if } u_{ij} = \phi_{ij}, \\ 0, & \text{if } \phi_{ij} < u_{ij} < \psi_{ij}, \\ [0, +\infty), & \text{if } u_{ij} = \psi_{ij}. \end{cases}$$

In indices,

$$\frac{1}{h^2}(4u_{ij} - u_{i-1,j} - u_{i+1,j} - u_{i,j-1} - u_{i,j+1}) + p_{ij}(u_{ij}) \ni f_{ij} \quad \text{for} \quad 1 \leqslant i, j \leqslant m;$$
$$u_{0j} = u_{m+1,j} = u_{i0} = u_{i,m+1} = 0.$$

SOR method for solving the above problem is:

$$Au + P(u) \ni f,$$

where u is the vector of the nodal values and $A = A^T > 0$ is SPD, P is a diagonal and monotone multivalued operator.

Let $A = D - L - L^T$, $\sigma \in (0,2)$, the SOR method is :

$$\left(\frac{1}{\sigma}D - L\right)u^{n+1} + P(u^{n+1}) \ni \left(\frac{1}{\sigma} - 1\right)Du^n + L^T u^n + f, \quad n = 0, 1, \ldots$$
$$u_i^{n+1} + \frac{\sigma}{a_{ii}}p_i(u_i^{n+1}) \ni (1-\sigma)u_i^n + \frac{\sigma}{a_{ii}}\left(\sum_{j>i} a_{ij}u_j^{n+1} + \sum_{j<i} a_{ij}u_j^n + f_i\right) \equiv F_i^n.$$

The details of the proposed algorithm can be seen in Table V as follows.

TABLE V: SOR method for solving problem (10)

Input: Mesh size m, function f, the values of ϕ and ψ.

Step 1: Initialize u randomly, set $k = m+2$, $h = 1/k$, $\sigma = 2/(1 + sin(pi * h))$ and $\varepsilon = h^2$.
Step 2: Set $u(1,:) = 0$, $u(k,:) = 0$, $u(:,1) = 0$, $u(:,k) = 0$, $sum = \varepsilon + 1$.
Step 3: While $sum >= \varepsilon$
% 1. calculate new iteration v:
for $i = 2:(k-1), j = 2:(k-1)$
$v = (1-\sigma)u_{ij} + \frac{\sigma}{4}(u_{i-1,j} + u_{i+1,j} + u_{i,j-1} + u_{i,j+1} + h^2 f_{ij})$.
End
define u_{ij}:
$$u_{ij} = \begin{cases} \phi_{ij}, & \text{if } v < \phi_{ij}, \\ v, & \text{if } \phi_{ij} \leqslant v \leqslant \psi_{ij}, \\ \psi_{ij}, & \text{if } v > \psi_{ij}. \end{cases}$$
calculate γ_{ij}:
$\gamma_{ij} = \frac{4}{h^2\sigma}(v - u_i j)$.
% 2. calculate residual vector r:
for $i = 2:(k-1), j = 2:(k-1)$
$r_{ij} = \frac{1}{h^2}(4u_{ij} - u_{i-1,j} - u_{i+1,j} - u_{i,j-1} - u_{i,j+1}) + \gamma_{ij} - f_{ij}$.
End
% 3. Compare:
$sum = 0$;
if $sum = (\sum_{1\leqslant i,j\leqslant m} h^2 r_{ij}^2)^{1/2} < \varepsilon$
break;
End
End

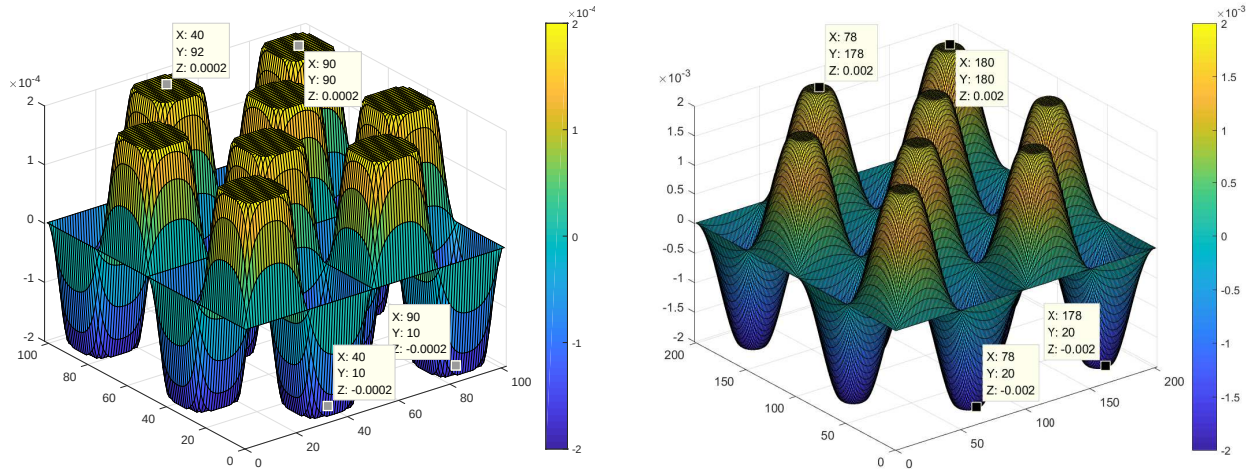
Output: u obtained by SOR method.

D. Experiments

We take some experiments to show the effectiveness of the above method. The selection of parameters are listed in the Table VI and the experimental results are illustrated in Fig.4, 5, 6.

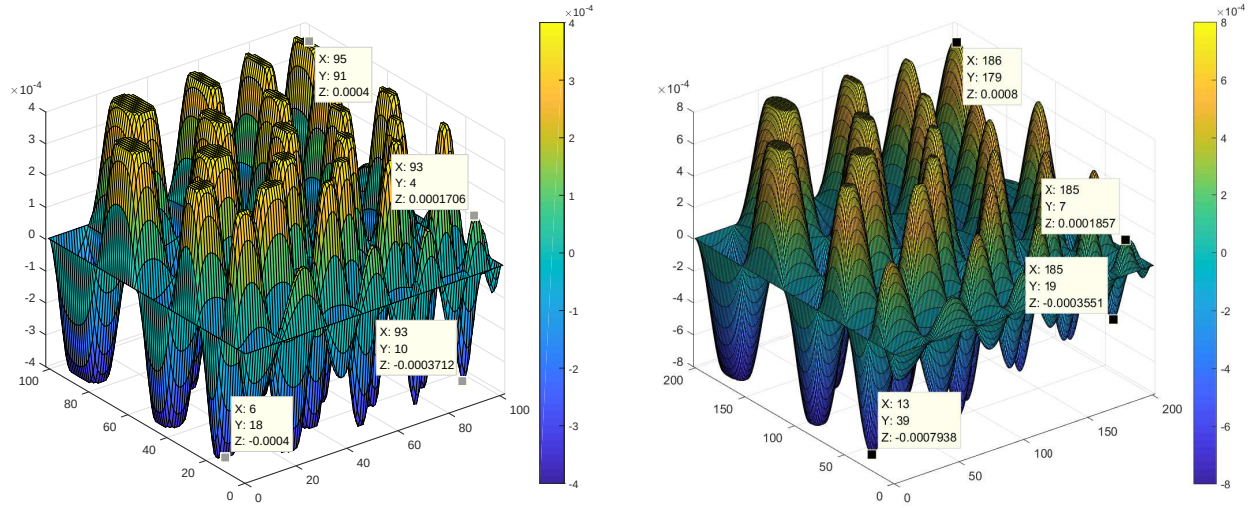
TABLE VI: The selection of parameters

	f	m	h	σ	ε	ϕ	ψ
Test 1	$f(t) = \sin 4\pi t_1 \sin 4\pi t_2$	100	$1/(m+2)$	$2/(1+sin(\pi h))$	h^2	-2×10^{-4}	2×10^{-4}
Test 2	$f(t) = \sin 4\pi t_1 \sin 4\pi t_2$	200	$1/(m+2)$	$2/(1+sin(\pi h))$	h^2	-2×10^{-3}	2×10^{-3}
Test 3	$f(t) = \sin 8\pi\sqrt{t_1} \sin 8\pi\sqrt{t_2}$	100	$1/(m+2)$	$2/(1+sin(\pi h))$	h^2	-4×10^{-4}	4×10^{-4}
Test 4	$f(t) = \sin 8\pi\sqrt{t_1} \sin 8\pi\sqrt{t_2}$	200	$1/(m+2)$	$2/(1+sin(\pi h))$	h^2	-8×10^{-4}	8×10^{-4}



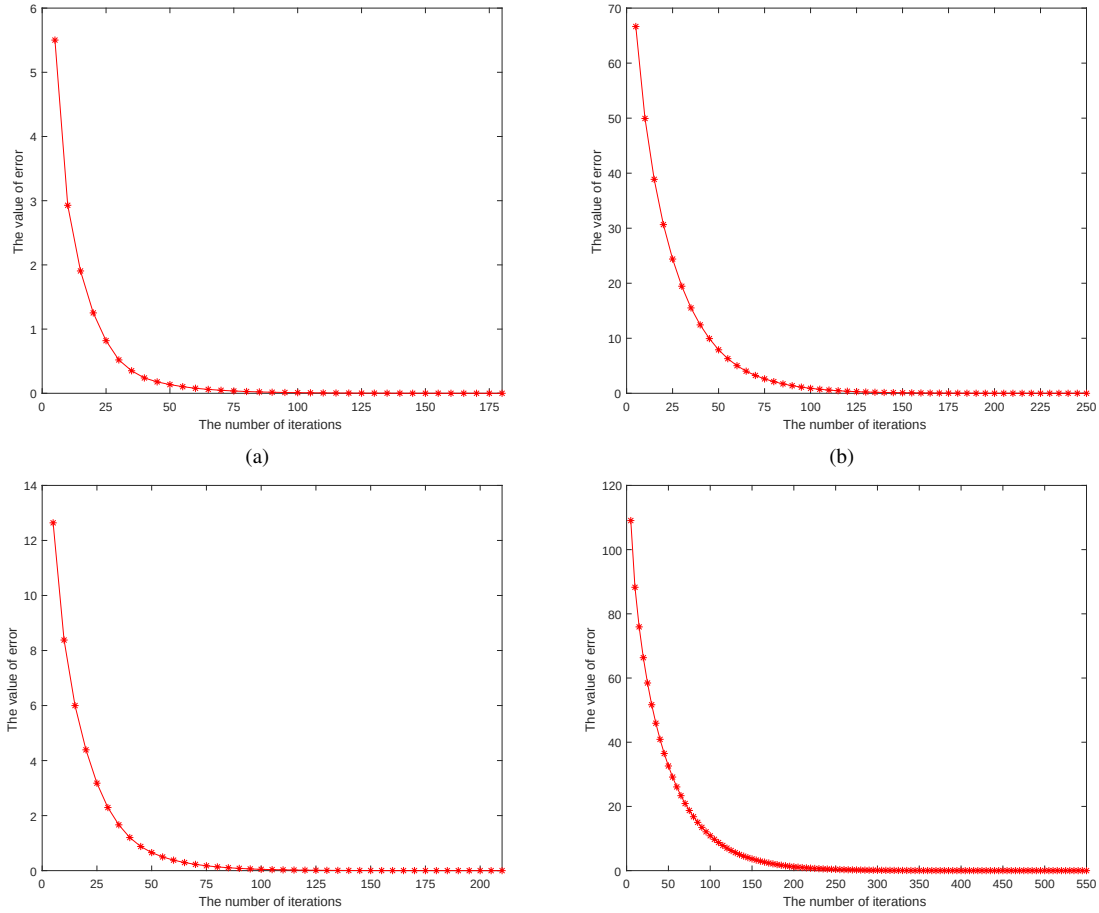
(a) (b)

Fig. 4: u obtained by SOR method (a) Test 1, (b) Test 2.



(a) (b)

Fig. 5: u obtained by SOR method (a) Test 3, (b) Test 4.



(a) (b) (c) (d)

Fig. 6: The convergence of the SOR method (a) Test 1, (b) Test 2, (c) Test 3, and (d) Test 4.

E. Conclusions

We solve the so-called two-sided obstacle problem by using the finite difference method. This problem contains all basic features of the mesh problem, in particular, mesh variational inequalities which approximate the differential problem: high dimension, big condition number and sparse matrix. As a result, we consider using SOR iterative method to solve this problem. Experimental results show the effectiveness and convergence of the used method. As can be seen in Fig.4 and Fig.5, with proper parameters ϕ and ψ, the constraints on u can be active. Moreover, the convergence of the used method, as can be seen in Fig.6, is fast, which indicates the efficiency of the used SOR method.

F. Matlab codes

```
clear
% Step 1:Initial
m = 200;
k = m+2;
h = 1 /k;
sigma = 2/(1 + sin(pi*h));
varepsilon = h^2;
u = randn(k,k);
gamma = zeros(k,k);
v = 0;
r = zeros(k,k);
% phi = zeros(k,k) - 2 * 10^(-3) * ones(k,k); %+ 0.2 * ones(k,k)
% psi = 2 * 10^(-3) * ones(k,k);
% phi = zeros(k,k) - 2 * 10^(-4) * ones(k,k); %+ 0.2 * ones(k,k)
% psi = 2 * 10^(-4) * ones(k,k);
% phi = zeros(k,k) - 4 * 10^(-4) * ones(k,k); %+ 0.2 * ones(k,k)
% psi = 4 * 10^(-4) * ones(k,k);
```

```
phi = zeros(k,k) - 8 * 10^(-4) * ones(k,k); %+ 0.2 * ones(k,k)
psi = 8 * 10^(-4) * ones(k,k);
u(1,:) = 0;
u(k,:) = 0;
u(:,1) = 0;
u(:,k) = 0;
sum = varepsilon+1;

while (sum ≥ varepsilon)
    % Step 2: Calculate the coordinates of the next iteration u
    for i = 2:(k-1)
        for j = 2:(k-1)
            v = (1-sigma)*u(i,j)+(sigma/4)*(u(i-1,j)+u(i+1,j)+u(i,j-1)+u(i,j+1)+h^2*f(i,j,h));
            if v < phi(i,j)
                u(i,j) = phi(i,j);
            elseif v > psi(i,j)
                u(i,j) = psi(i,j);
            else
                u(i,j) = v;
            end
            gamma(i,j) = (4*(v-u(i,j))) / (h^2*sigma);
        end
    end

    %Step 3: Calculate residual
    for i = 2:(k-1)
        for j = 2:(k-1)
            r(i,j) = (1/h^2)*(4*u(i,j)-u(i-1,j)-u(i+1,j)-u(i,j-1)-u(i,j+1))-f(i,j,h)+gamma(i,j);
        end
    end

    %Step 4: Compare
    sum = 0;
    for i = 2:(k-1)
        for j = 2:(k-1)
            sum = sum + h^2*r(i,j)^2;
        end
    end
    sum = sum^(1/2);
end
surf(u);

function value = f(i,j,h)
    %value = sin(4*pi*i*h) * sin(4*pi*j*h);
    value = sin(8*pi*(i*h)^(1/2)) * cos(8*pi*j*h*(j*h)^(1/2));
end
```

III. Lagrange multipliers method for a quadratic with inequality constraint

A. Preliminary

The method of Lagrange multipliers:

Consider the following minimization problem:

$$\min\{\theta(u) \mid \mathcal{A}u = b,\ u \in \mathcal{U}\} \tag{11}$$

where $\mathcal{U} \subset \Re^n$ is a convex set, $\theta(u)$: $\Re^n \to \Re$ is a convex function, $\mathcal{A} \in \Re^{m\times n}$ and $b \in \Re^m$.

The Lagrangian function of the above problem is as follows:

$$L(u,\lambda) = \theta(u) - \lambda^T(\mathcal{A}u - b) \tag{12}$$

If there exists $\omega^* = (u^*, \lambda^*) \in \mathcal{U} \times \Re^m$ such that

$$L_{\lambda\in\Re^m}(u^*,\lambda) \leqslant L(u^*,\lambda^*) \leqslant L_{u\in\mathcal{U}}(u,\lambda^*) \tag{13}$$

ω^* is the saddle point of the Lagrangian function L, where u^* is the optimal solution of problem (11), and λ^* is the corresponding optimal Lagrange multiplier.

The problem (13) can be rewritten as follows:

$$\begin{cases} u^* \in \mathcal{U}, & L(u,\lambda^*) - L(u^*,\lambda^*) \geqslant 0; \quad \forall u \in \mathcal{U} \\ \lambda^* \in \Re^m, & L(u^*,\lambda^*) - L(u^*,\lambda) \geqslant 0; \quad \forall \lambda \in \Re^m \end{cases}$$

or in the form:

$$\omega^* \in \Omega, \quad \theta(u) - \theta(u^*) + (\omega - \omega^*)F(\omega^*) \geqslant 0, \quad \forall \omega \in \Omega$$

where $\omega = \begin{pmatrix} u \\ \lambda \end{pmatrix}$, $F(\omega) = \begin{pmatrix} -\mathcal{A}^T\lambda \\ \mathcal{A}u - b \end{pmatrix}$, $\Omega = \mathcal{U} \times \Re^m$.

Augmented Lagrangian method:

The Augmented Lagrangian function for solving the problem (11) is as follows:

$$L_\beta(u,\lambda) = \theta(u) - \lambda^T(\mathcal{A}u - b) + \frac{\beta}{2}||\mathcal{A}u - b||^2 \tag{14}$$

where β is a given parameter.

The $k+1$-th iterations of the Augmented Lagrangian method (ALM) for solving the problem (11) is as follows:

$$(ALM) \quad \begin{cases} u^{k+1} = \arg\min\left\{\mathcal{L}_\beta(u,\lambda^k) \mid u \in \mathcal{U}\right\} \\ \lambda^{k+1} = \lambda^k - \beta(\mathcal{A}u^{k+1} - b) \end{cases}$$

Since the iterations can start as long as there is λ^k. In this case, we call u an intermediate variable, and λ is a core variable.

Alternating direction method of multipliers:

In particular, a variant of the standard ALM that uses partial updates (similar to the Gauss-Seidel method for solving linear equations) is known as the alternating direction method of multipliers (ADMM).

Suppose that the problem (11) has a separable structure: $\theta(u) = \theta_1(x) + \theta_2(y)$, $\mathcal{A} = (A, B)$ and $\mathcal{U} = \mathcal{X} \times \mathcal{Y}$, then we have the following problem:

$$\min\{\theta_1(x) + \theta_2(y) \mid \mathcal{A}x + By = b,\ x \in \mathcal{X}, y \in \mathcal{Y}\} \tag{15}$$

The advantage of using ADMM to solve this problem is that we can use the separable structure of the problem. The $k+1$-th iterations of the ADMM for solving the problem (15) is as follows:

$$(ADMM) \quad \begin{cases} x^{k+1} = \arg\min\left\{\mathcal{L}_\beta(x,y^k,\lambda^k) \mid x \in \mathcal{X}\right\} \\ y^{k+1} = \arg\min\left\{\mathcal{L}_\beta(x^{k+1},y,\lambda^k) \mid y \in \mathcal{Y}\right\} \\ \lambda^{k+1} = \lambda^k - \beta(Ax^{k+1} + By^{k+1} - b) \end{cases} \tag{16}$$

The first two subproblem of problem (16) are as follows:

$$\begin{cases} \min_{x\in\mathcal{X}}\left\{\theta_1(x) + \frac{\beta}{2}||Ax + (By^k - b - \frac{1}{\beta}\lambda^k)||^2\right\} \\ \min_{y\in\mathcal{Y}}\left\{\theta_2(y) + \frac{\beta}{2}||By + (Ax^{k+1} - b - \frac{1}{\beta}\lambda^k)||^2\right\} \end{cases}$$

Saddle point problem:

The linear systems of the form

$$\begin{pmatrix} A & B_1^T \\ B_2 & -C \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} f \\ g \end{pmatrix} \quad \text{or} \quad \mathcal{A}u = b, \tag{17}$$

where $A \in \Re^{n\times n}$, $B_1, B_2 \in \Re^{mtimesn}$, $C \in \Re^{m\times m}$, with $n \geqslant m$, is called generalized saddle point problem.

In the case that A is symmetric positive semidefinite and $C = O$, we have a symmetric linear system of the form:

$$\begin{pmatrix} A & B^T \\ B & O \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} f \\ g \end{pmatrix} \tag{18}$$

The system arises as the first-order optimality conditions for the following equality-constrained quadratic programming problem:

$$\min J(x) = \frac{1}{2}x^T Ax - f^T x \quad \text{subject to} \quad Bx = g \tag{19}$$

In this case, the variable y represents the vector of Lagrange multipliers. Any solution (x^*, y^*) of (18) is a saddle point for the Lagrangian

$$\mathcal{L}(x, y) = \frac{1}{2}x^T Ax - f^T x + (Bx - g)^T y \tag{20}$$

hence the name 'saddle point problem' given to (18). Recall that a saddle point is a point $(x^*, y^*) \in \Re^{n+m}$ that satisfies

$$\mathcal{L}(x^*, y) \leqslant \mathcal{L}(x^*, y^*) \leqslant \mathcal{L}(x, y^*) \quad \text{for any} \quad x \in \Re^n \quad \text{and } y \in \Re^m \tag{21}$$

or, equivalently,

$$\min_x \max_y \mathcal{L}(x, y) = \mathcal{L}(x^*, y^*) = \max_y \min_x \mathcal{L}(x, y) \tag{22}$$

The assumption that A is nonsingular may appear to be rather restrictive, since A is singular in many applications. However, one can use augmented Lagrangian techniques to replace the original saddle point system with an equivalent one having the same solution but in which the $(1, 1)$ block A is now nonsingular.

Uzawa methods:

To solve problem (18), we consider Uzawa's method, which enjoys considerable popularity in fluid dynamics, especially for solving the (steady) Stokes problem.

Starting with initial guess x_0 and y_0, Uzawa's method consists of the following coupled iteration:

$$\begin{cases} Ax^{k+1} = f - B^T y^k \\ y^{k+1} = y^k + \tau(Bx^{k+1} - g) \end{cases}$$

where $\tau > 0$ is a relaxation parameter.

Preconditioned Uzawa-type method consists of the following coupled iteration:

$$\begin{cases} x^{k+1} = x^k + A^{-1}(f - Ax^k - B^T y^k) \\ y^{k+1} = y^k + \tau Q^{-1}(Bx^{k+1} - g) \end{cases}$$

where Q is a symmetric positive definite matrix.

Inexact Uzawa-type (PIU) method consists of the following coupled iteration:

$$\begin{cases} x^{k+1} = x^k + \omega P^{-1}(f - Ax^k - B^T y^k) \\ y^{k+1} = y^k + \tau Q^{-1}(Bx^{k+1} - g) \end{cases}$$

where $\omega > 0$, $\tau > 0$, P and Q are symmetric positive definite matrices, and Q is an approximate (or preconditioning) matrix of the approximated Schur complement matrix $B^T P^{-1} B + C$.

SOR-like method consists of the following coupled iteration:

$$\begin{cases} x^{k+1} = (1 - \omega)x^k + \omega A^{-1}(f - Ax^k - B^T y^k) \\ y^{k+1} = y^k + \omega Q^{-1}(Bx^{k+1} - g) \end{cases}$$

where Q is a nonsingular symmetric matrix.

B. Problem definition

Considering the following minimization problem:

$$J(u) = \frac{1}{2}\int_0^1 u'^2(t)dt - \int_0^1 b(t)\,u(t)dt$$
$$\text{on the set} \quad \{u(t) \in H_0^1(0,1) : |u'(t)| \leqslant 1 \ \text{ for } t \in (0,1)\}$$

where $H_0^1(0,1)$ is the space of the functions which are defined on $(0,1)$, have (generalized) derivatives of 1-st order and such that $u(0) = u(1) = 0$.

We approximate this problem by the finite difference method to obtain an optimization problem with inequality constraints in the next section.

C. Solution of Dirichlet boundary value problem for Poisson equation

Let $\{t_i = i\,h,\ i = 0,\dots,n+1;\ (n+1)\,h = 1\}$ be a uniform grid with mesh size $h > 0$ on the segment $[0,1]$. We denote $u_i = u(t_i)$ with $u_0 = u_{n+1} = 0$ and $b_i = b(t_i)$.

Finite difference scheme for the differential minimization problem under consideration is

$$\begin{gathered}\text{find} \quad \min_{u\in K} F(u) = \frac{1}{2}\left(\frac{u_1^2}{h^2} + \sum_{i=1}^{n-1}\left(\frac{u_{i+1}-u_i}{h}\right)^2 + \frac{u_n^2}{h^2}\right) - \sum_{i=1}^{n} b_i u_i, \\ \text{on the set} \quad K = \left\{\frac{|u_i - u_{i-1}|}{h} \leqslant 1\ \forall i = 1,\dots,n+1\right\}\end{gathered} \tag{23}$$

(We underline that $u_0 = u_{n+1} = 0$ for all mesh functions u).

Defining the matrices

$$L = h^{-1}\begin{pmatrix} 1 & 0 & 0 & \dots & 0 & 0 \\ -1 & 1 & 0 & \dots & 0 & 0 \\ 0 & -1 & 1 & \dots & 0 & 0 \\ \dots & \dots & \dots & \dots & \dots & \dots \\ 0 & 0 & 0 & \dots & -1 & 1 \\ 0 & 0 & 0 & \dots & 0 & -1 \end{pmatrix} \in \mathbb{R}^{(n+1)\times n}.$$

and $\mathcal{A} = L^T\,L \in \mathbb{R}^{n\times n}$, problem (23) can be written as

$$\begin{gathered}\text{find} \quad \min\{\frac{1}{2}(\mathcal{A}u,u) - (b,u)\} \\ \text{on the set} \quad K,\end{gathered} \tag{24}$$

with symmetric and positive definite matrix $\mathcal{A}$. (Above $(.,.)$ means the inner product in $\mathbb{R}^n$).

Note, that this problem is equivalent also to the variational inequality

$$(\mathcal{A}u, v-u) \geqslant (b, v-u)\ \forall v \in K,$$

and the inclusion

$$\mathcal{A}u \ni b.$$

We can apply, for example, a gradient projection method to solve problem (24) However, projecting onto the set

$$K = \left\{\frac{|u_i - u_{i-1}|}{h} \leqslant 1\ \forall i = 1,\dots,n+1\right\}$$

is a problem that cannot be solved by a direct method and requires the use of an additional iterative method. The complexity of implementation of such methods is comparable to the methods of solving the original problem.

How to solve problem (23)**?** The most reasonable approach is to use **Lagrange multipliers method**, which is described below.

Let $\theta(p)$ be the indicator function of the set

$$K_p = \{p \in \mathbb{R}^{(n+1)} : |p_i| \leqslant 1\ \forall i = 1,2,\dots,n+1\}.$$

Using the introduced notations we write problem (23) in the form: find minimum of the function

$$\frac{1}{2}\|Lu\|^2 - (b,u) + \theta(Lu),\ u \in \mathbb{R}^n. \tag{25}$$

Let us define a new vector $p = Lu$, then (25) is equivalent to

$$\min_{Lu=p} \left(\frac{1}{2}\|p\|^2 - (b,u) + \theta(p) \right).$$

We will solve this problem by Lagrange multipliers method. Namely, let Lagrange function be defined as

$$\mathcal{L}(u,p,\lambda) = \frac{1}{2}\|p\|^2 - (b,u) + \theta(p) + (Lu - p, \lambda).$$

Then saddle point of $\mathcal{L}$ is a triple (u,p,λ), satisfying the system

$$\begin{aligned} &\frac{\partial \mathcal{L}}{\partial u}(u,p,\lambda) = 0 \Leftrightarrow L^T\lambda = b, \\ &\partial_p \mathcal{L}(u,p,\lambda) \ni 0 \Leftrightarrow p + \partial\theta(p) - \lambda \ni 0, \\ &\frac{\partial \mathcal{L}}{\partial \lambda}(u,p,\lambda) = 0 \Leftrightarrow Lu = p. \end{aligned} \tag{26}$$

It is well-known, that if there exists saddle point (u,p,λ) of Lagrange function $\mathcal{L}$, when u is a solution of (25) and $p = Lu$.

System (26) can be written as

$$\begin{pmatrix} 0 & 0 & L^T \\ 0 & I & -I \\ L & -I & 0 \end{pmatrix} \begin{pmatrix} u \\ p \\ \lambda \end{pmatrix} + \begin{pmatrix} 0 \\ \partial\theta(p) \\ 0 \end{pmatrix} \ni \begin{pmatrix} b \\ 0 \\ 0 \end{pmatrix}, \tag{27}$$

where I is the unit $(n+1)\times(n+1)$ matrix.

Remark 1: Using the notations

$$\tilde{A} = \begin{pmatrix} 0 & 0 & L^T \\ 0 & I & -I \\ L & -I & 0 \end{pmatrix}, \ \tilde{P} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & \partial\theta & 0 \\ 0 & 0 & 0 \end{pmatrix}, \ \tilde{u} = \begin{pmatrix} u \\ p \\ \lambda \end{pmatrix}, \ \tilde{b} = \begin{pmatrix} b \\ 0 \\ 0 \end{pmatrix},$$

we can write (27) in "traditional" form

$$\tilde{A}\tilde{u} + \tilde{P}(\tilde{u}) \ni \tilde{b}.$$

Now operator $\tilde{P}$ is diagonal, matrix $\tilde{A}$ is symmetric. But it is not definite positive, it has both positive and negative eigenvalues. Such kind of matrices is called "saddle matrices". Problem (27) is a constrained saddle point problem.

Now, let

$$A = \begin{pmatrix} 0 & 0 \\ 0 & I \end{pmatrix}, \ B = \begin{pmatrix} -L & I \end{pmatrix}, \ \partial\varphi = \begin{pmatrix} 0 & 0 \\ 0 & \partial\theta \end{pmatrix}, f = \begin{pmatrix} b \\ 0 \end{pmatrix}, x = \begin{pmatrix} u \\ p \end{pmatrix}. \tag{28}$$

Then system (27) reads as

$$\begin{pmatrix} A & -B^T \\ -B & 0 \end{pmatrix} \begin{pmatrix} x \\ \lambda \end{pmatrix} + \begin{pmatrix} \partial\varphi(u) \\ 0 \end{pmatrix} \ni \begin{pmatrix} f \\ 0 \end{pmatrix}. \tag{29}$$

Matrix A in (27) is **degenerate** and this prevents the usage of the iterative solution methods.

To avoid this deficiency of matrix A, we will do several equivalent transformations of (27) and propose a new Lagrange function.

Let us introduce once again the artificial constraint $Lu = p$ and rewrite (25) in the following equivalent form:

$$\min_{Lu=p} \left(\frac{1}{4}\|Lu\|^2 + \frac{1}{4}\|p\|^2 - (b,u) + \theta(p) \right). \tag{30}$$

The corresponding Lagrange function is

$$\mathcal{L}(u,p,\lambda) = \frac{1}{4}\|Lu\|^2 + \frac{1}{4}\|p\|^2 - (b,u) + \theta(p) + (Lu - p, \lambda)$$

and its saddle-point satisfies the system

$$\begin{pmatrix} 0.5\,L^TL & 0 & L^T \\ 0 & 0.5\,I & -I \\ L & -I & 0 \end{pmatrix} \begin{pmatrix} u \\ p \\ \lambda \end{pmatrix} + \begin{pmatrix} 0 \\ \partial\theta(p) \\ 0 \end{pmatrix} \ni \begin{pmatrix} b \\ 0 \\ 0 \end{pmatrix}. \tag{31}$$

Once again we get a particular case of problem (29), now with positive definite matrix

$$A_3 = \frac{1}{2} \begin{pmatrix} L^TL & 0 \\ 0 & I \end{pmatrix}.$$

The details of the proposed algorithm can be seen in Table VII as follows.

TABLE VII: Uzawa-type method for solving problem (23)

Input: Mesh size m, function b, the values of τ.

Step 1: Initialize $u_i = \lambda_i = p_i = 0$, for $\forall i \in \{1, 2, \ldots, k\}$, set $k = m + 2$, $h = 1/k$, and $\varepsilon = h^2$.
Step 2: Set $u(1) = 0$, $u(k) = 0$, and $sum = sum2 = \varepsilon + 1$.
Step 3: While $sum >= \varepsilon$
% 1. calculate new iteration u:
$sum2 = \varepsilon + 1$.
While $sum2 >= \varepsilon$
for $i = 2 : (k-1)$
$u_i = \frac{1}{2}(u_{i-1} + u_{i+1}) + h^2 b_i - h * (\lambda_i - \lambda_{i-1})$.
End
for $i = 2 : (k-1)$
$r_i = \frac{1}{h^2}(2u_i - u_{i-1} - u_{i+1}) - 2(b_i - (\lambda_i - \lambda_{i+1})/h)$
End
$sum2 = 0$.
for $i = 2 : (k-1)$
$sum2 = sum2 + h^2 r_i^2$.
End
$sum2 = \sqrt{sum2}$.
End
% 2. calculate p:
for $i = 2 : k$:
if $\lambda_i > 1/2$, $p_i = 1$; elseif $\lambda_i < -1/2$, $p_i = -1$; else, $p_i = 2\lambda_i$.
End
% 3. calculate λ:
$temp = \lambda$.
for $i = 2 : k$:
$\lambda_i = \lambda_i + \tau(\frac{1}{h}(u_i - u_{i-1}) - p_i)$.
End
% 4. calculate sum:
$sum = ||\lambda^{k-1} - \lambda^k||_2 = ||temp - \lambda||_2$
End

Output: u obtained by Uzawa-type method.

D. Experiments

We take some experiments to show the effectiveness of the above Uzawa-type method. The selection of parameters are listed in the Table VIII, the value of σ vs. the number of iterations are listed in the Table IX–XI and the experimental results are illustrated in Fig.7–9.

TABLE VIII: The selection of parameters

	b	m	h	ε	τ
Experiment A	$b = \begin{cases} 100, & x < 1/4, \\ 192(x-1/4)^2 - 96(x-1/4) + 12, & 1/4 \leqslant x \leqslant 3/4, \\ 100, & x > 3/4. \end{cases}$	100	$1/(m+2)$	h^2	$1/3$
Experiment B	$b = \pi sin(\pi x)$	100	$1/(m+2)$	h^2	$1/3$
Experiment C	$b = \pi^2 sin(\pi x)$	100	$1/(m+2)$	h^2	$1/3$

TABLE IX: The value of τ vs. the number of iterations, where b is taken from Experiment A.

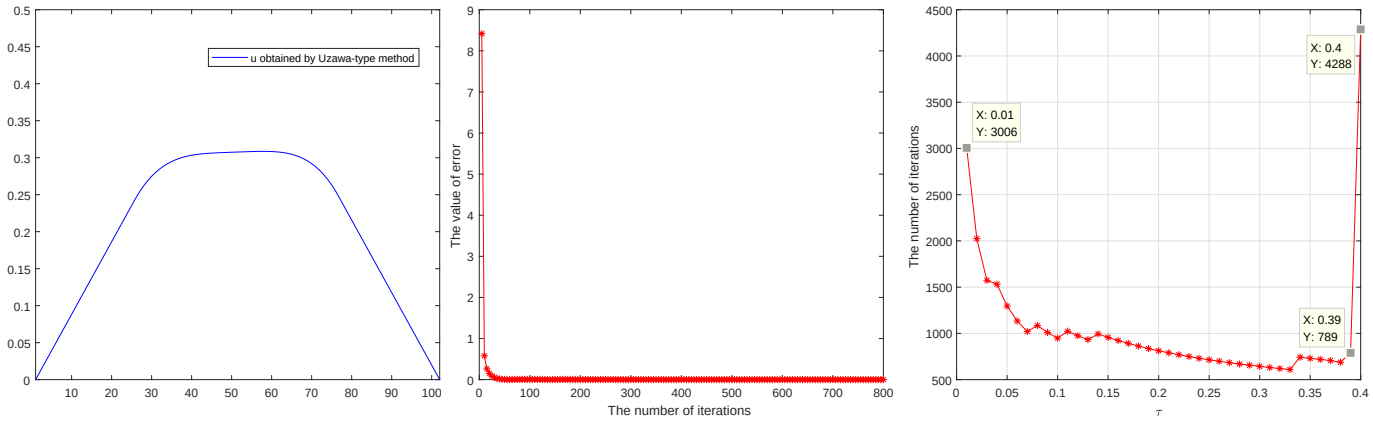
τ	0.01	0.02	0.03	0.04	0.05	0.06	0.07	0.08	0.09	0.10	0.11	0.12	0.13
iterations	3006	2025	1574	1532	1296	1134	1020	1085	1010	949	1023	975	933
τ	0.14	0.15	0.16	0.17	0.18	0.19	0.20	0.21	0.22	0.23	0.24	0.25	0.26
iterations	994	957	923	892	863	836	812	790	769	750	731	714	699
τ	0.27	0.28	0.29	0.30	0.31	0.32	0.33	0.34	0.35	0.36	0.37	0.38	0.39
iterations	683	669	656	644	631	620	609	743	730	719	706	688	789

TABLE X: The value of τ vs. the number of iterations, where b is taken from Experiment B.

τ	0.01	0.02	0.03	0.04	0.05	0.06	0.07	0.08	0.09	0.10	0.11	0.12	0.13	0.14
iterations	2727	2527	2521	2516	2512	2509	2506	2504	2502	2501	2499	2497	2496	2495
τ	0.15	0.16	0.17	0.18	0.19	0.20	0.21	0.22	0.23	0.24	0.25	0.26	0.27	0.28
iterations	2494	2493	2493	2491	2490	2489	2488	2487	2485	2484	2481	2477	2469	2457
τ	0.29	0.30	0.31	0.32	0.33	0.34	0.35	0.36	0.37	0.371	0.372	0.373	0.3740	0.375
iterations	2441	2423	2403	2380	2355	2328	2299	2268	2269	2244	2537	2489	2270	3655

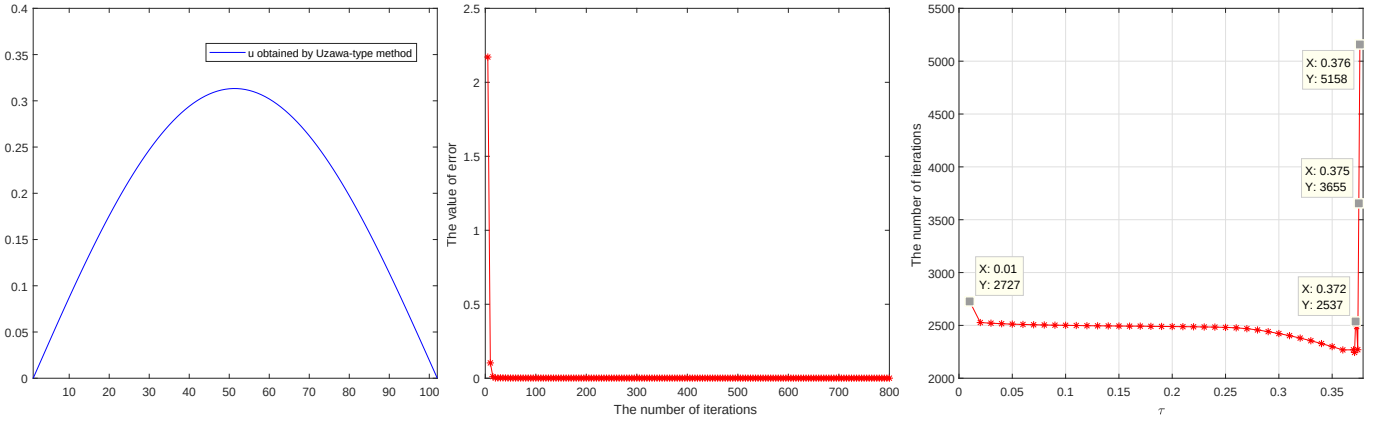
TABLE XI: The value of τ vs. the number of iterations, where b is taken from Experiment C.

τ	0.001	0.002	0.003	0.004	0.005	0.006	0.007	0.008	0.009	0.010	0.020	0.030	0.040	0.050	0.060
iterations	37459	3903	4428	3842	4433	4005	4551	4263	4764	4503	5168	5436	5052	5143	4976
τ	0.070	0.080	0.090	0.100	0.110	0.120	0.130	0.140	0.150	0.160	0.170	0.180	0.190	0.200	0.210
iterations	4601	4547	4282	4057	4074	3896	3739	3599	3474	3359	3256	3160	3073	3146	3067
τ	0.220	0.230	0.240	0.250	0.260	0.270	0.280	0.290	0.300	0.310	0.320	0.330	0.340	0.350	0.360
iterations	2993	2924	2859	2799	2741	2688	2636	2461	2417	2376	2336	2298	2262	2227	2193
τ	0.370	0.380	0.390	0.400	0.410	0.420	0.430	0.440	0.450	0.451	0.452	0.453	0.454	0.455	0.456
iterations	2162	2130	2101	2071	2140	1988	2020	1817	2879	2757	5113	17433	25385	320313	306120



(a) (b) (c)

Fig. 7: The parameters are selected from Experiment A (a) u obtained by Uzawa-type method, (b) The convergence of the Uzawa-type method (c) the number of iterations vs. the parameter τ



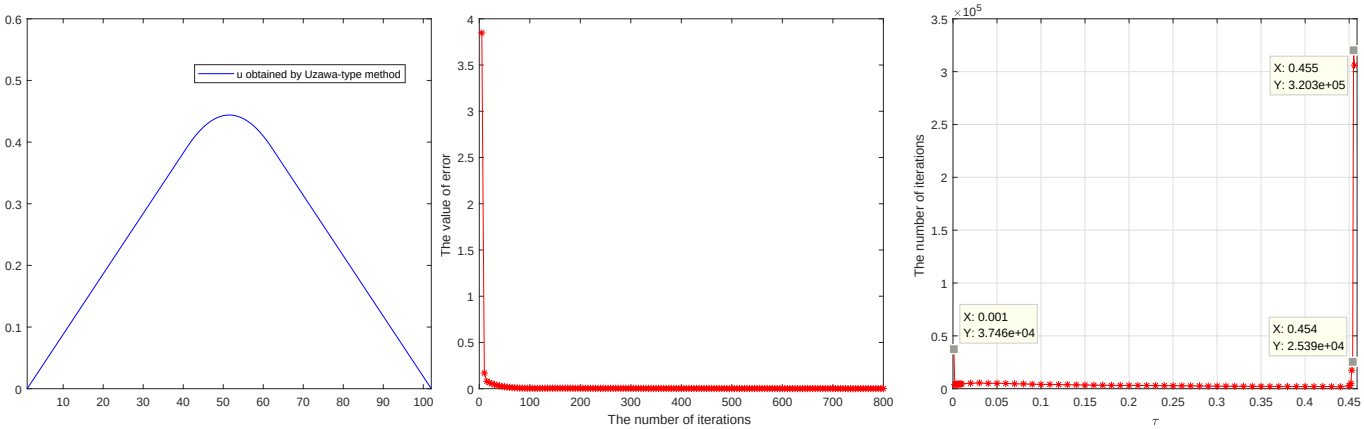
(a) (b) (c)

Fig. 8: The parameters are selected from Experiment B (a) u obtained by Uzawa-type method, (b) The convergence of the Uzawa-type method (c) the number of iterations vs. the parameter τ

E. Conclusions

We solve an optimization problem with equality and inequality constraints by using the finite difference method. Although a gradient projection method can solve the obtained problem, projecting onto the given set is a problem that cannot be solved by a direct method. Therefore, we use Lagrange multipliers method to solve this problem. The experiments show the effectiveness and efficiency of the taken method. Obviously, the value of the parameter τ is sensitive to the convergence of the used method.

As can be seen in Table IX and Fig.7 (c), the optimal parameter τ in the experiment with the parameter selection of Experiment A is approximated to 0.33, which fits the to the theoretical value $\frac{1}{3}$.



(a) (b) (c)

Fig. 9: The parameters are selected from Experiment C (a) u obtained by Uzawa-type method, (b) The convergence of the Uzawa-type method (c) the number of iterations vs. the parameter τ

F. Matlab codes

```
%Initial
clear,clc;
n = 100;
k = n+2;
h = 1/k;
tau = 1/3;
lambda = zeros(1,k);
varepsilon = h^2;
u = zeros(1,k);
residual = zeros(1,k);
u(1) = 0;
u(k) = 0;
p = zeros(1,k);
varepsilon2 = h^2;
sum = varepsilon + 1;
Tsum = [];

while (sum ≥ varepsilon)
    % Step 1: Calculate new u
    clear sum
    sum2 = varepsilon2+1;
    while (sum2 ≥ varepsilon2)
        % Step 1_1: Calculate new u
        for i = 2:(k-1)
            u(i) = (1/2)*(u(i-1)+u(i+1))+h^2*b(i,h)-h*(lambda(i)-lambda(i+1));
        end
        %Step 1_2: Calculate residual
        for i = 2:(k-1)
            residual(i) = (1/h^2)*(2*u(i)-u(i-1)-u(i+1))-(2*b(i,h)-2*(lambda(i)-lambda(i+1))/h);
        end
        %Step 1_3: Compare
        sum2 = 0;
        for i = 2:(k-1)
            sum2 = sum2 + h^2*residual(i)^2;
        end
        sum2 = sum2^(1/2);
    end

    %Step 2: Calculate p
    for i = 2:k
        if lambda(i) > (1/2)
            p(i) = 1;
        elseif lambda(i) < (-1/2)
```

```
            p(i) = -1;
        else
            p(i) = 2*lambda(i);
        end
    end

    %Step 3: Calculate lambda
    temp = lambda;
    for i = 2:(k)
        lambda(i) = lambda(i)+tau*((u(i)-u(i-1))/h-p(i));
    end

    %Step 4: Compare
    sum = norm(temp(2:k)-lambda(2:k),2);
    clear temp
    sum = sum^(1/2);
    Tsum = [Tsum sum];
end
%plot(Tsum(1:5:811),'r-*')
plot(1:k,u,'b')
legend(  'u obtained by Uzawa-type method' );
axis([1 k 0 0.5])

function value = b(i,h)
    t = i*h;
    if t < 1/4
        value = 100;
    elseif t > 3/4
        value = 100;
    else
        value = 192*(t-1/4)^2-96*(t-1/4)+12;
    end
    %value = pi*sin(pi*t);    %for Experiment B
    %value = pi^2*sin(pi*t);  %for Experiment C
end
```